to determine whether to continue with the delete or return a `403 Forbidden` response.

Listing 25.3 Using resource authorization to protect a minimal API endpoint

```
app.MapDelete("recipe/{id}", async (
    int id, RecipeService service,
    IAuthorizationService authService,                ❶
    ClaimsPrincipal user) =>                          ❷
{
    var recipe = await service.GetRecipe(id);         ❸
    var result = await authService.AuthorizeAsync(    ❹
        user, recipe, "CanManageRecipe");             ❹

    if (!result.Succeeded)                            ❺
    {                                                 ❺
        return Results.Forbid();                      ❺
    }                                                 ❺

    await service.DeleteRecipe(id);                   ❻
    return Results.NoContent();                       ❻
});
```

❶ Injected to perform resource-based authorization

❷ The HttpContext.User claims principal for the request

❸ Fetches the recipe to access

❹ Performs resource-based authorization, passing in the user, resource, and the policy name

❺ If authorization failed, returns 403 Forbidden

❻ If authorization succeeded, executes the endpoint as normal

As is common when you start adding functionality, the logic at the heart of the endpoint has become a bit muddled as the endpoint has grown. There are several possible approaches you could take now:

- *Do nothing*. The logic isn't *that* confusing, and this is only one endpoint. This may be a good approach initially but can become problematic if the logic is duplicated across multiple endpoints.